SDD Playbook — Guía de bolsillo

> ***Toolkit:*** *sdd-toolkit* · ***Basado en:*** *WORKFLOW v19* · ***Autor:*** *Martin Bortagaray* ***Propósito:*** *Guía paso a paso imprimible de cada flujo del proceso SDD con los comandos del plugin de Claude Code. Cada sección es autocontenida (pensada para imprimirse como ficha).*

> ***Nota de invocación:*** *todos los comandos se invocan con el prefijo del plugin:* `/sdd-toolkit:<comando>` *(ej.* `/sdd-toolkit:sdd-discovery` *). Por brevedad, en este documento los escribo sin el prefijo. Tipeá* `/sdd` *en Claude Code para autocompletarlos.*

---

0 · Referencia rápida

Los 12 comandos

Comando	Fase	Qué hace
`sdd-feature`	**2→6 + commit**	**Modo Flow (por defecto): ciclo completo, solo frena en 4 gates.** Feature nueva o modificación.
`sdd-discovery`	2	Te interroga para extraer decisiones. No decide por vos.
`sdd-spec`	3	Redacta el draft de una spec **nueva**.
`sdd-modify-spec`	3 (sobre feature existente)	Edita una spec **existente** (Implemented/As-built).
`sdd-adversarial-spec`	4	Ataca la spec buscando gaps (subagente, contexto limpio).
`sdd-verify`	5→6	Semáforo VERDE/AMARILLO/ROJO antes de codear.
`sdd-codegen`	6	Genera código: toda la feature con plan único (por defecto) o una capa puntual.
`sdd-adversarial-code`	6	Ataca el código contra la spec (subagente). Una pasada por feature, o por capa.
`sdd-bugfix`	§9	Trata un bug con trazabilidad (clasifica A/B/C).
`sdd-defer`	§8.7	Registra un pendiente diferido en `DEBT.md`. Captura barata, no frena la sesión.
`sdd-debt-review`	§8.7	Revisa `DEBT.md`: actualiza el estado de cada pendiente (implementado / pendiente / promovido / descartado).
`sdd-commit`	—	Commit inteligente con convención SDD.

Las 5 reglas no negociables

1. La IA **nunca toma decisiones de producto** sin preguntar. Aunque parezca obvio.
2. La IA **marca cada decisión por defecto** en una sección clara, para validarla.
3. **Leo la spec entera** antes de aprobarla. No "una mirada".
4. **Toda spec pasa por pasada adversaria** antes de aprobarse. No se saltea.
5. Si una spec aprobada estaba mal: **subo versión y documento**. No corrijo en silencio.

Jerarquía de gobierno (la capa superior gobierna)

```
Setup foundacional  →  Feature Spec  →  Código
```

El código nunca contradice la spec; la spec nunca contradice el setup foundacional.

Árbol de decisión — ¿qué tipo de cambio tengo?

```
¿El sistema NO hace lo que la spec dice?
   └─ Sí → BUG → sdd-bugfix   (Tipo A: spec ok / Tipo B: spec mal)

¿Hace lo correcto, pero quiero agregar/cambiar funcionalidad?
   ├─ ¿Parte de una feature existente?    → sdd-feature <ID> "cambio"   ← caso dominante en gitelli
   └─ ¿Capacidad nueva e independiente?    → sdd-feature "necesidad"     (spec nueva)

¿El cambio se repite en varias features o es arquitectónico?
   └─ Primero el setup foundacional (sdd/foundation/), después la spec
```

Flujo 0 · Modo Flow — `sdd-feature` (camino por defecto)

> *Una sola invocación recorre todo el ciclo y **solo me frena en 4 gates**. Sirve para feature nueva y para modificación (le paso el ID de la spec + el cambio). Si se corta, lo vuelvo a invocar con el ID y retoma desde donde quedó.*

`sdd-feature "necesidad en 1-2 líneas"` · `sdd-feature <ID> "qué cambiar"`

G1 — Preguntas en lote (Fases 2+3, spec-first)

- ☐ La IA ya escribió el draft en `sdd/specs/<ID>.md` ; lo que es decisión mía viene como `[DECIDIR-N]` con opciones y recomendación. En una modificación, acá también confirmo el tier.
- ☐ Respondo `ok` (acepto las recomendadas) · `ok salvo 3:B, 5:<texto>` · `ayudame con N` . Nada de "es estándar".

G2 — Aprobación de spec (Fases 4→5)

- ☐ La revisión unificada (adversaria + verify, subagente) ya corrió y aplicó lo que no era decisión mía.
- ☐ Leo la spec entera (Regla 3). La ficha me da: cambios de la revisión (aplicados / descartados), decisiones nuevas, §14 a validar, semáforo.
- ☐ Las 3 decisiones clave, el trade-off y la parte débil las nombro yo (la ficha no me las da). Si mis respuestas cambian la spec, veo el diff.
- ☐ Respondo `leí, apruebo` (un `ok` suelto no aprueba) → **Approved** + INDEX al día (lo hace la IA). En features **S**, acá también viene el plan (G3 fusionado).

G3 — Plan único (Fase 6)

- ☐ Una tabla para todas las capas: requerimiento → capa → archivos → test. `plan ok` .

(Sin gates: tests de CA primero → capas 1→4 con gate automático tests + typecheck → una revisión de código sobre el diff completo → bloqueantes corregidos, no bloqueantes a `DEBT.md` .)

G4 — Prueba manual → commit

- ☐ Sigo el plan de prueba (pasos, datos, resultado esperado). Valido en lote las decisiones técnicas por defecto.
- ☐ `ok` → commitea (`feat(<ID>): …`). `ok push` → además pushea. Si falla, lo digo y vuelve a código.

Me interrumpe fuera de los gates solo si: hay una decisión de producto, el tier sube (válvula de escape), la spec choca con el foundation, los tests siguen rojos tras 2 intentos, o el semáforo da ROJO.

Flujo A · Feature nueva (camino manual / modo profundo)

> *Para features **L**, o cuando quiero discovery sección por sección y control capa por capa. Recorre las 6 fases invocando cada comando.*

Fase 1 — Definir necesidad

- [ ] Escribo el problema en 1-2 líneas + notas crudas (restricciones, dependencias). Sin formato.

Fase 2 — Discovery → `sdd-discovery "necesidad"`

- [ ] Respondo las preguntas: decisión concreta / "no decidido aún" / "ayudame a pensarlo".
- [ ] No esquivo decisiones ("es estándar", "lo vemos después" → el comando me frena).
- [ ] Output: documento de respuestas + pendientes + dependencias.

Antes de Fase 3: resuelvo todos los `[PENDIENTE]` (consulto a agrónomo/socio/experto si hace falta) y valido cada decisión por defecto.

Fase 3 — Draft de spec → `sdd-spec [discovery] [ID]`

- [ ] La IA redacta llenando el template. No "mejora" mis decisiones, las traduce.
- [ ] Si aparece un gap de producto → me frena. Lo resuelvo.
- [ ] Output: `sdd/specs/<ID>.md` en estado **Draft**. Actualizo INDEX.

Fase 4 — Pasada adversaria → `sdd-adversarial-spec [ID] 1`

- [ ] Corre en subagente (contexto limpio). Devuelve hallazgos por 10 categorías.
- [ ] Clasifico: sólidos para iterar / zona gris / descartables (estilo, casos teóricos).
- [ ] Proceso los aceptados, documento en sub-sección 10.2, subo versión. Estado → **Review**.
- [ ] **Máximo 2 pasadas.** Si la pasada 2 sigue mostrando problemas de fondo: parar y repensar.

Fase 5 — Aprobación (lectura crítica con cabeza fresca)

- [ ] Pre-requisitos: cero `[PENDIENTE]` / `TBD`, dudas externas resueltas y escritas, ≥1 pasada adversaria hecha.
- [ ] **Señales de aprobar con criterio:** puedo nombrar las 3 decisiones clave, identifico un trade-off, sé cuál es la parte más débil y por qué la acepto.
- [ ] **Señales de aprobar por inercia (PARAR):** "se ve bien" sin más, no recuerdo qué se decidió, estoy cansado/apurado.
- [ ] Si OK → estado **Approved**, registro fecha + versión.

Fase 5→6 — Verificación → `sdd-verify [ID]`

- [ ] 🟢 VERDE → sigo. 🟡 AMARILLO → decido conscientemente y registro. 🔴 ROJO → resuelvo bloqueantes.

Fase 6 — Codegen por capas → `sdd-codegen [ID] [capa]` (capas 1→2→3→4)

Por cada capa:

- [ ] La IA genera **primero la tabla comparativa** (spec vs. implementación). La reviso.
- [ ] Confirmo "tabla ok, generá". Recién ahí escribe código + tests.
- [ ] Reviso decisiones técnicas por defecto. Sin sobre-ingeniería.
- [ ] `sdd-adversarial-code [ID] [capa]` (subagente) → proceso bloqueantes ya, no-bloqueantes a `DEBT.md`.
- [ ] Tests de la capa pasan. Recién entonces paso a la capa siguiente.
- [ ] Capa 2+: cargar schema real de la DB. Capa 1: migraciones **append-only**.

Cierre → `sdd-commit`

- [ ] **Un solo commit por feature completa y verificada** (no por capa). Ver Flujo D para deploy.

Flujo B · Modificar feature existente (spec Implemented/As-built)

> *Caso dominante en gitelli. Cambio de producto sobre una feature que ya existe. Re-entra al ciclo **acotado al delta**.*
>
> ***Camino por defecto:*** `sdd-feature <ID> "qué cambiar"` *— clasifica el tier, hace las preguntas del delta en lote junto con la propuesta de tier, y sigue el Flujo 0 con los pasos que el tier exige (T1 = carril rápido). Los pasos de abajo son el camino manual.*

Paso 0 — Clasifico (ver árbol de decisión en §0)

- [ ] ¿Es bug? → Flujo C. ¿Capacidad nueva independiente? → Flujo A. ¿Se repite en varias features? → primero el foundation.
- [ ] Si es agregar/cambiar funcionalidad de esta feature → sigo acá.

Paso 1 — Leo la spec existente con ojo crítico (Regla 3)

- [ ] ¿Refleja bien el comportamiento actual? **Ojo con las as-built:** suelen tener huecos.
- [ ] Si la base está floja, completarla es parte de este cambio (espíritu de Regla 5).

Paso 2 — Discovery del delta → `sdd-discovery "qué agrego/cambio"`

- [ ] Acotado al cambio, no re-discovery de toda la feature. Resuelvo pendientes y valido decisiones por defecto.

Paso 3 — Modifico la spec → `sdd-modify-spec [ID] [discovery]`

- [ ] Edita **solo las secciones afectadas** (no reescribe). Sube versión + entrada de changelog.
- [ ] Si toca el modelo de datos → en código será una **nueva migración append-only**.
- [ ] Si el delta es de producto no resuelto, o choca con el foundation → me frena.

Paso 4 — Adversaria + verificación → `sdd-adversarial-spec` → (apruebo) → `sdd-verify`

- [ ] La Regla 4 **no se relaja** por ser "solo un cambio".

Paso 5 — Codegen del delta → `sdd-codegen` (solo las capas que el cambio toca)

- [ ] Si el delta no afecta el modelo de datos, no se regenera Capa 1.
- [ ] `sdd-adversarial-code` por capa tocada → `sdd-commit` .

Flujo C · Bug

> *Un bug no rompe "spec antes que código". Cambia la velocidad, no la trazabilidad.*

Paso 1 — Clasifico ANTES de tocar código → `sdd-bugfix "descripción"`

Tipo	Definición	Qué corrijo
A — Implementación	El código no cumple lo que la spec dice. La spec estaba bien.	Solo el código.
B — Spec	La spec no contempló el caso / era ambigua. El código hizo lo que decía.	Primero la spec (nueva versión), luego el código.
C — Negocio	El sistema cumple la spec, pero el negocio cambió de opinión.	No es bug → es feature/modificación (Flujo A o B).

> *Si dudo entre A y B, casi siempre es **B**.*

Paso 2 — Artefacto `sdd/bugs/bugfix-NNN.md` (estado **Abierto**)

- [ ] Descripción (observado vs esperado), reproducción, root cause, criterio de aceptación testeable, severidad, tipo.

Paso 3 — Según severidad

- [ ] **Crítico** (prod caída / dato corrupto): fix mínimo ya, artefacto Abierto el mismo día, completar en 24h.
- [ ] **Alto/Medio/Bajo:** proceso completo sin atajos. Si **Tipo B** → **actualizo la spec primero** (sube versión).

Paso 4 — Test de regresión (regla de cierre)

- [ ] Escribo el test **antes** del fix: debe **fallar** con el código actual.
- [ ] Genero el fix **mínimo** (sin refactor, sin mejoras de paso).
- [ ] El test **pasa** y queda en la suite. Lo referencio en el `bugfix-NNN.md`.

Paso 5 — Cierre

- [ ] Estado **Cerrado** solo con: fix verificado + test en verde + **prueba manual** + spec actualizada si era Tipo B. Mi `ok` a la prueba manual autoriza el commit (`fix(bugfix-NNN):`), sin re-confirmar.

> ***Velocidad (v19):*** *`sdd-bugfix` presenta clasificación + root cause + test + fix propuesto **en un solo mensaje**; con mi `ok` avanza solo hasta la prueba manual.*

Flujo C-bis · Pendientes diferidos (DEBT.md)

> *Para ideas, deuda técnica o decisiones que surgen a mitad de sesión y **decido diferir a propósito**. Regla: "lo dejo registrado" obliga a una escritura concreta en `sdd/DEBT.md` — no es una frase suelta.*

Registrar → `sdd-defer "qué difiero" [tipo]`

- [ ] Tipo: `deuda-tecnica` (refactor / hallazgo no bloqueante) · `idea-producto` (feature charlada) · `decision-diferida`.
- [ ] **No es bug** (→ Flujo C) ni **hallazgo bloqueante** (se corrige ya, no se difiere).
- [ ] Agrega entrada `DEBT-NNN` estado **Abierto** + fila en la tabla de índice. **No frena** lo que estabas haciendo.

Revisar / cosechar → `sdd-debt-review [filtro opcional]`

- [ ] Recorre las entradas no terminales; propone estado nuevo con evidencia del repo, yo confirmo.
- [ ] Estados: **Abierto** (sigue pendiente) → **Resuelto** (ya se implementó) · **Promovido** (pasó a spec / `ROADMAP.md` / `bugfix-NNN.md`, anota destino) · **Descartado** (anota por qué).
- [ ] Sincroniza la tabla de índice y sube versión de `DEBT.md`. Las `Abierto` son las que reclaman acción.

Promover un pendiente maduro: `sdd-discovery` (feature), entrada de `ROADMAP.md`, o `sdd-bugfix`.

Flujo D · Commit y deploy (ramas)

> *Modelo GitHub Flow simplificado.* ***Nadie pushea directo a `main`, ni yo.***

```
main        → producción. Siempre estable.
staging     → validación del cliente.
feature/*   → una rama por spec (feature/SPEC-ID).
hotfix/*    → fixes urgentes de producción.
```

Durante la feature

- [ ] Al iniciar Fase 6: `git checkout main && git pull` → `git checkout -b feature/SPEC-ID`.
- [ ] **Un solo commit** cuando la feature está completa y verificada (no por capa). Formato: `feat(SPEC-ID): descripción`.

Deploy

- [ ] A staging: `git checkout staging && git pull && git merge feature/SPEC-ID && git push` → deploy manual → cliente valida.
- [ ] Cliente aprueba → a producción: `git checkout main && git pull && git merge feature/SPEC-ID && git push` → deploy manual.
- [ ] `git branch -d feature/SPEC-ID`. Actualizo INDEX: spec **Approved** → **Implemented**.

Hotfix

- [ ] `git checkout -b hotfix/desc` desde main → fix → merge a main **y** a staging → borrar rama.

Flujo E · Instalar / actualizar el plugin

> *Se hace **una vez por máquina** (scope user → todos los proyectos). Requiere el CLI `claude`.*

Instalar (terminal / PowerShell)

```
claude plugin marketplace add "C:\Users\mrtbr\CODE\sdd-toolkit"   # local (dev)
# o desde GitHub:  claude plugin marketplace add martin-bortagaray/sdd-toolkit
claude plugin install sdd-toolkit@sdd-toolkit                     # --scope user por default
claude plugin list                                                # verificar
```

- [ ] **Reiniciar** Claude Code para que carguen los comandos. Verificar con `/sdd`.

Actualizar (después de editar el toolkit)

```
# 1. Si agregaste/cambiaste comandos: subí "version" en .claude-plugin/plugin.json (ej. 1.0.0 → 1.1.0)
# 2. git commit && git push     (el marketplace de directorio snapshotea el commit)
claude plugin marketplace update sdd-toolkit       # 3. refresca el marketplace
claude plugin update sdd-toolkit@sdd-toolkit       # 4. recopia el plugin instalado
```

- [ ] **Crítico — bump de versión:** el plugin instalado es una **copia cacheada**; `plugin update` se fija en el número de `version`, no en el contenido. Si no subís la versión, dice "already at the latest version" y **no toma los comandos nuevos** aunque reinicies. Para cambios de solo contenido (editar un prompt/comando existente, sin agregar comandos) el bump no es estrictamente necesario, pero ante la duda, subila.
- [ ] **Reiniciar** la sesión de Claude Code al final. Verificar con `/sdd`.

Recordatorios — mis zonas de riesgo

- Soy ansioso: tiendo a avanzar rápido y cerrar incomodidad procesando poco.
- Cierro feedback demasiado rápido ("ya entendí, avancemos") en vez de procesarlo.
- Sub-especifico criterios de aceptación y casos borde. Releer y aplicar la taxonomía.
- Esquivo decisiones con "es estándar" / "lo definimos después".
- No releo antes de cerrar. **Trabajar cansado/de noche aumenta el riesgo de aprobar mal.**

Antipatrones a no cometer: vibe coding · spec retroactiva · saltar la adversaria "porque es chica" · modificar spec sin frenar la generación · generar capas sin gate entre ellas (tests + typecheck) · confirmaciones que no deciden nada · aceptar código que "funciona" sin verificar que cumple spec · commitear un fix sin su `bugfix-NNN.md` · cerrar un bug sin test de regresión.

> *Documento de referencia. Para el detalle operativo completo, ver `workflow/WORKFLOW.md`.*